

PORTADA

Técnicas de Programación

Unidad 4 — Archivos, Errores/Excepciones y Pruebas: JUnit y Pruebas Unitarias

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Si nadie más que tú verifica que tu código funciona, cada cambio nuevo es una apuesta a que no rompiste nada de lo viejo.



TRANSICIÓN

Empieza la Unidad 4

En las últimas dos sesiones trabajaste con **excepciones** para manejar errores en tiempo de ejecución, y con **archivos de texto**, **CSV y JSON** para leer y escribir datos, además de generar reportes en **PDF**. Toda esa lógica —parsear un archivo, calcular un total, decidir si lanzar una excepción— necesita algo que la verifique de forma repetible: hoy aprendes a escribir **pruebas unitarias** con JUnit 5, en vez de probar todo a mano cada vez que cambias una línea.

CONCEPTO

¿Qué es una prueba unitaria?

Una **prueba unitaria** (*unit test*) es un pequeño programa que verifica **automáticamente** que una unidad de código — típicamente un método— se comporta como se espera, sin que una persona tenga que ejecutar la aplicación y revisar el resultado a simple vista.

- Le das al método unas **entradas conocidas**.
- Comparas el **resultado real** contra el **resultado esperado**.
- La prueba misma decide, sin intervención humana, si pasó o falló.

Es la diferencia entre correr el programa y hacer clic para "ver si funciona", y tener un script que lo confirma en milisegundos, cada vez que lo necesites.

CONCEPTO

¿Por qué automatizar pruebas?

Beneficio	En la práctica
Detectar regresiones temprano	Si cambias un método y rompes otro que dependía de él, la prueba falla al instante — no meses después, en producción.
Documentar el comportamiento esperado	Una prueba bien nombrada dice, en código ejecutable, qué se supone que hace el método — más confiable que un comentario que puede quedar desactualizado.
Dar confianza para refactorizar	Puedes reescribir la implementación interna de un método (mejorar su Big-O, por ejemplo) sabiendo que si las pruebas siguen pasando, el comportamiento externo no cambió.

CONCEPTO

JUnit 5: el framework de pruebas de Java

JUnit es el framework estándar de la industria para escribir y ejecutar pruebas unitarias en Java. Se agrega como una dependencia de Maven en el `pom.xml` del proyecto:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

Con `scope: test`, la librería solo está disponible al compilar/ejecutar pruebas — no viaja con el programa final. Las clases de prueba viven, por convención, en `src/test/java`, en paralelo a `src/main/java`.

CONCEPTO

La anotación @Test

La anotación `@Test` marca un método como una prueba: JUnit lo detecta y lo ejecuta automáticamente, sin que nadie lo llame desde un `main` .

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculadoraTest {

    @Test
    void deberiaSumarDosNumerosPositivos() {
        Calculadora calc = new Calculadora();
        int resultado = calc.sumar(2, 3);
        assertEquals(5, resultado);
    }
}
```

Convención: la clase de prueba se llama igual que la clase probada + el sufijo `Test` .

CONCEPTO

Preparar y limpiar: @BeforeEach / @AfterEach

Muchas pruebas necesitan preparar el mismo objeto antes de cada verificación. En vez de repetir ese código en cada método de prueba, `@BeforeEach` lo ejecuta automáticamente **antes de cada** `@Test`, y `@AfterEach` **después de cada uno** (útil para liberar recursos, cerrar archivos, etc.).

```
class CalculadoraTest {  
    private Calculadora calc;  
  
    @BeforeEach  
    void inicializar() {  
        calc = new Calculadora();  
    }  
  
    @Test  
    void deberiaSumarDosNumerosPositivos() {  
        assertEquals(5, calc.sumar(2, 3));  
    }  
}
```


CONCEPTO

Una sola vez por clase: @BeforeAll / @AfterAll

A diferencia de @BeforeEach / @AfterEach (que corren en **cada** prueba), @BeforeAll y @AfterAll se ejecutan **una sola vez** por clase de prueba: antes de la primera prueba y después de la última. Se usan para recursos costosos de preparar (una conexión a base de datos, cargar un archivo grande) que no vale la pena repetir en cada prueba.

```
@BeforeAll
static void configurarUnaVez() {
    // debe ser static: se ejecuta antes de crear
    // cualquier instancia de la clase de prueba
    System.out.println("Preparando recursos compartidos...");
}
```

Importante: @BeforeAll y @AfterAll deben ser métodos `static`, porque JUnit los invoca antes de tener siquiera una instancia de la clase.

CONCEPTO

Aserciones: comparando lo esperado con lo real

Una **aserción** (*assertion*) es la instrucción que realmente decide si la prueba pasa o falla: compara un valor **esperado** contra el valor **real** que produjo el código, y lanza un error de prueba si no coinciden. Todas viven en la clase `Assertions` (paquete `org.junit.jupiter.api`).

Método	Verifica
<code>assertEquals(esperado, actual)</code>	Que dos valores sean iguales
<code>assertTrue(condicion)</code> / <code>assertFalse(condicion)</code>	Que una condición booleana sea verdadera/falsa
<code>assertNull(valor)</code> / <code>assertNotNull(valor)</code>	Que una referencia sea (o no) <code>null</code>

CONCEPTO

Probar que se lanza una excepción: `assertThrows`

Retomando lo visto en la sesión de **excepciones**: si un método debe lanzar una excepción en cierto caso (por ejemplo, saldo insuficiente), `assertThrows` verifica exactamente eso — que se lanzó la excepción **correcta**, no simplemente que el programa "falló".

```
@Test
void deberiaLanzarExcepcionCuandoSaldoEsInsuficiente() {
    CuentaBancaria cuenta = new CuentaBancaria(100.0);

    assertThrows(SaldoInsuficienteException.class, () -> {
        cuenta.retirar(500.0);
    });
}
```

El segundo argumento es una **expresión lambda** (vista en la Unidad 2) que envuelve el código que se espera que falle. Si no lanza esa excepción exacta, la prueba falla.

CONCEPTO

Agrupar verificaciones: `assertAll`

Cuando una prueba necesita verificar varias condiciones sobre el mismo resultado, `assertAll(...)` las agrupa y ejecuta **todas**, aunque alguna falle — así el reporte muestra de una vez cuáles fallaron, en vez de detenerse en la primera.

```
@Test
void deberiaCrearEstudianteConDatosCorrectos() {
    Estudiante e = new Estudiante("Ana", 20);

    assertAll(
        () -> assertEquals("Ana", e.getNombre()),
        () -> assertEquals(20, e.getEdad()),
        () -> assertNotNull(e.getId())
    );
}
```

Con aserciones simples (sin `assertAll`), la primera que falla detiene la prueba y oculta el resto.

CONCEPTO

Ciclo de vida: aislamiento entre pruebas

Cada método `@Test` de una clase corre sobre una **instancia nueva** de esa clase de prueba. JUnit crea un objeto distinto por cada prueba, ejecuta `@BeforeEach`, corre el `@Test`, y luego `@AfterEach` — así una prueba nunca puede "heredar" un estado modificado por la prueba anterior.

Esto es lo que hace posible que las pruebas sean **independientes entre sí**: puedes ejecutarlas en cualquier orden, o solo una de ellas, y el resultado no cambia.

CONCEPTO

El patrón Arrange-Act-Assert

La forma más clara de estructurar el cuerpo de una prueba es en tres pasos, siempre en el mismo orden:

1. **Arrange** (preparar): crear los objetos y datos de entrada necesarios.
2. **Act** (ejecutar): llamar al método que se está probando.
3. **Assert** (verificar): comparar el resultado real contra el esperado.

```
@Test
void deberiaAplicarDescuentoDelDiez() {
    // Arrange
    Factura factura = new Factura(100.0);
    // Act
    double total = factura.aplicarDescuento(0.10);
    // Assert
    assertEquals(90.0, total);
}
```

CONCEPTO

Nombres descriptivos, no genéricos

El nombre del método de prueba debe describir, en lenguaje natural, **qué comportamiento verifica** — así, cuando falla en el reporte, se entiende el problema sin abrir el código.

Nombre genérico (evitar)	Nombre descriptivo (preferir)
test1()	deberiaSumarDosNumerosPositivos()
testRetiro()	deberiaLanzarExcepcionCuandoSaldoEsInsuficiente()
testVacio()	deberiaRetornarListaVacíaCuandoNoHayResultados()

INTERACCIÓN

Escriban la prueba que falta

En parejas: dado este método, escriban un método `@Test` completo (con nombre descriptivo, siguiendo Arrange-Act-Assert) que verifique que lanza la excepción correcta cuando la edad es negativa.

```
public class Estudiante {  
    public Estudiante(String nombre, int edad) {  
        if (edad < 0) {  
            throw new IllegalArgumentException("La edad no puede ser negativa");  
        }  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```

8 minutos · usen `assertThrows`

CONCEPTO

Buenas prácticas al escribir pruebas

- **Una prueba, un comportamiento:** si un método de prueba verifica cinco cosas distintas sin relación, y falla, no queda claro cuál se rompió. Prefiere varias pruebas pequeñas y enfocadas (o `assertAll` cuando de verdad son parte del mismo caso).
- **Pruebas independientes:** ninguna prueba debe depender de que otra se ejecutó antes, ni del orden en que JUnit las corra — el aislamiento de instancia (visto antes) ya ayuda, pero también evita compartir estado mutable entre pruebas (como campos `static`).
- **Sin lógica compleja dentro de la prueba:** evita `if` , ciclos o cálculos elaborados en el cuerpo de una prueba — una prueba con su propia lógica puede tener sus propios bugs, y entonces ¿quién prueba la prueba?

SÍNTESIS

Lo que hay que llevarse de la sesión

1. Una prueba unitaria verifica automáticamente el comportamiento de un método, sin intervención manual.
2. `@Test` marca una prueba; `@BeforeEach` / `@AfterEach` preparan y limpian antes/después de cada una; `@BeforeAll` / `@AfterAll` (static) lo hacen una sola vez por clase.
3. `assertEquals` , `assertTrue/False` , `assertNull/NotNull` , `assertThrows` y `assertAll` son las aserciones base de `Assertions` .
4. Cada `@Test` corre en una instancia nueva de la clase — las pruebas están aisladas entre sí.
5. Arrange-Act-Assert + nombres descriptivos hacen que una prueba se lea como documentación del comportamiento esperado.



CIERRE

Antes de la próxima sesión

- Escribe pruebas `@Test` para al menos dos métodos de un proyecto propio, aplicando Arrange-Act-Assert y nombres descriptivos.
- Piensa en un método que lance una excepción y practica verificarlo con `assertThrows`.

Próxima sesión: librerías para análisis y visualización de datos (Apache Commons CSV/Math, JFreeChart) y principios de software inclusivo — cierre de la Unidad 4, antes del Laboratorio 2.

